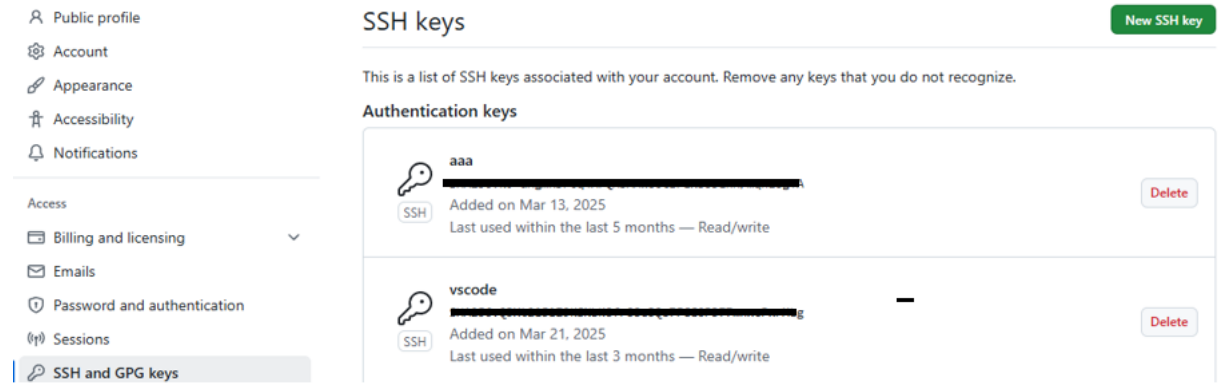
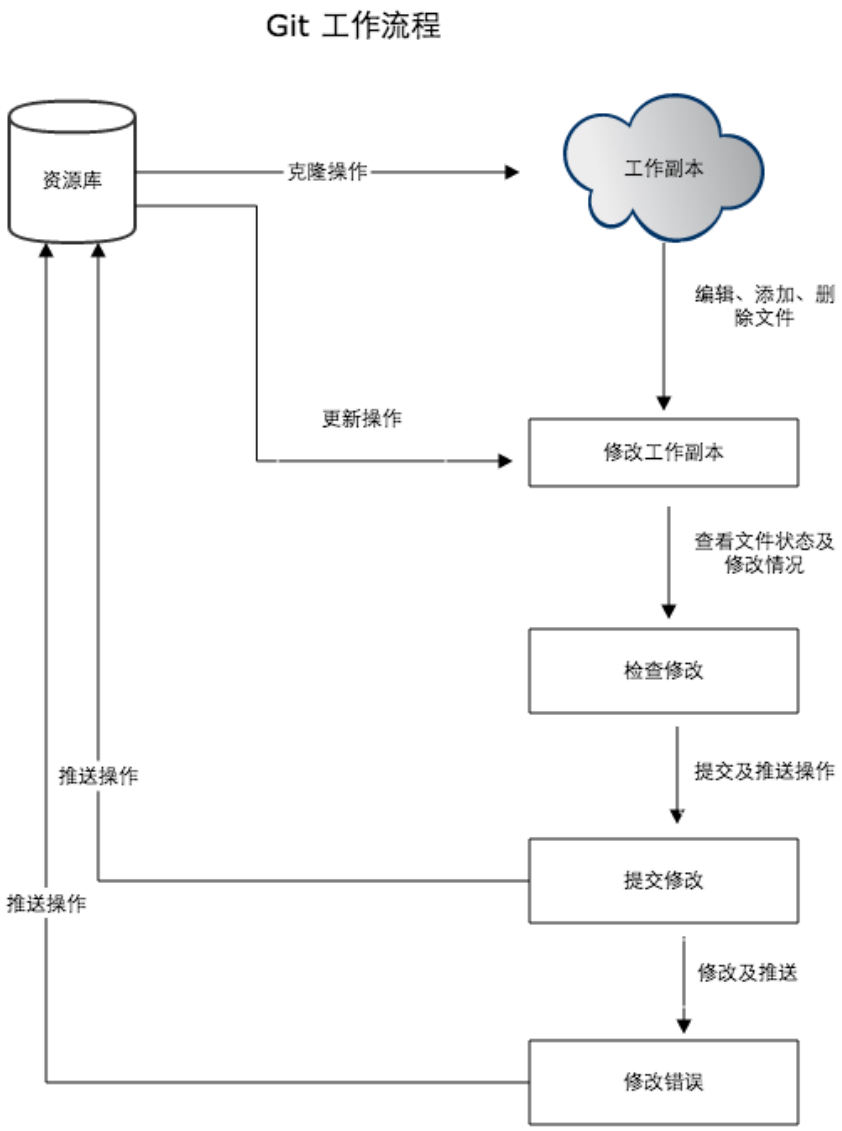
# Git 分享

## 1.Git 的安装与配置

<table>
<tr><td>Git 下载</td><td colspan="2">https://git-scm.com/download/win</td></tr>
<tr><td>用户信息配置</td><td colspan="2">git config --global user.name "runoob"<br>git config --global user.email test@runoob.com</td></tr>
<tr><td>查看配置</td><td>git config --list</td><td>http.postbuffer=2M<br>user.name=runoob<br>user.email=test@runoob.com</td></tr>
<tr><td rowspan="2">Git 连接，生成 SSH 密钥</td><td rowspan="2">ssh-keygen -t rsa -b 4096 -C<br>"your.email@example.com"</td><td>查看公钥：cat ~/.ssh/id_rsa.pub</td></tr>
<tr><td>查看私钥：cat ~/.ssh/id_rsa</td></tr>
<tr><td colspan="3">配置公钥后当前机器获得对代码仓库的操作权限<br>私钥在对代码操作时 身份验证</td></tr>
<tr><td colspan="3"> 
</td></tr>
</table>

## 2.Git 工作流程




整体使用步骤

创建仓库 - 克隆仓库 - 本地修改（增删改）- 推送远程

本地修改：工作区、暂存区、本地仓库

推送远程：远程仓库

重要操作：切取分支、推送远程、合并分支、合并冲突

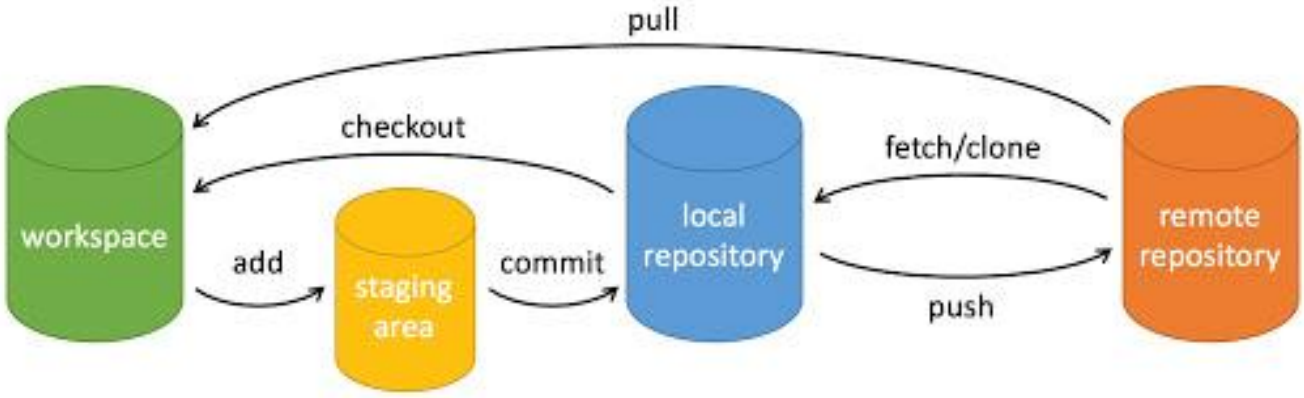
## 3.Git 使用-初始化 Git 仓库

| | |
|---|---|
| 当前项目能够使用 git 命令对代码进行操作 | git init |
| 指定目录使用 git 命令对代码进行操作 | git init project_name |

## 从代码仓库拷贝项目

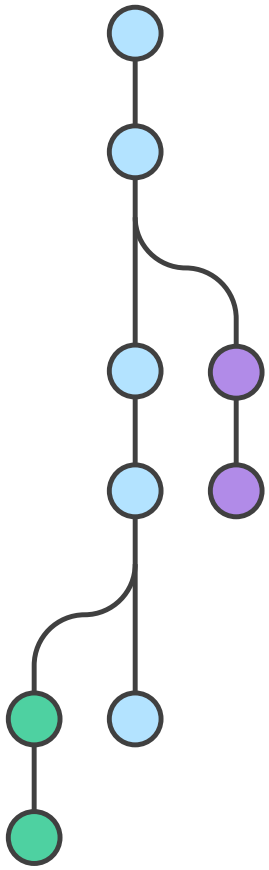
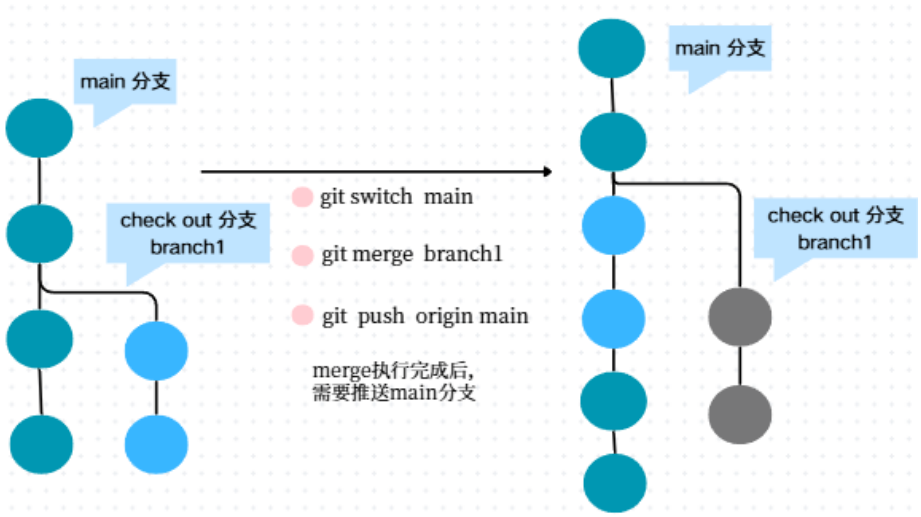
| | |
|---|---|
| 克隆仓库 | git clone <repo> |
| 克隆到指定的目录 | git clone <repo> <directory> |

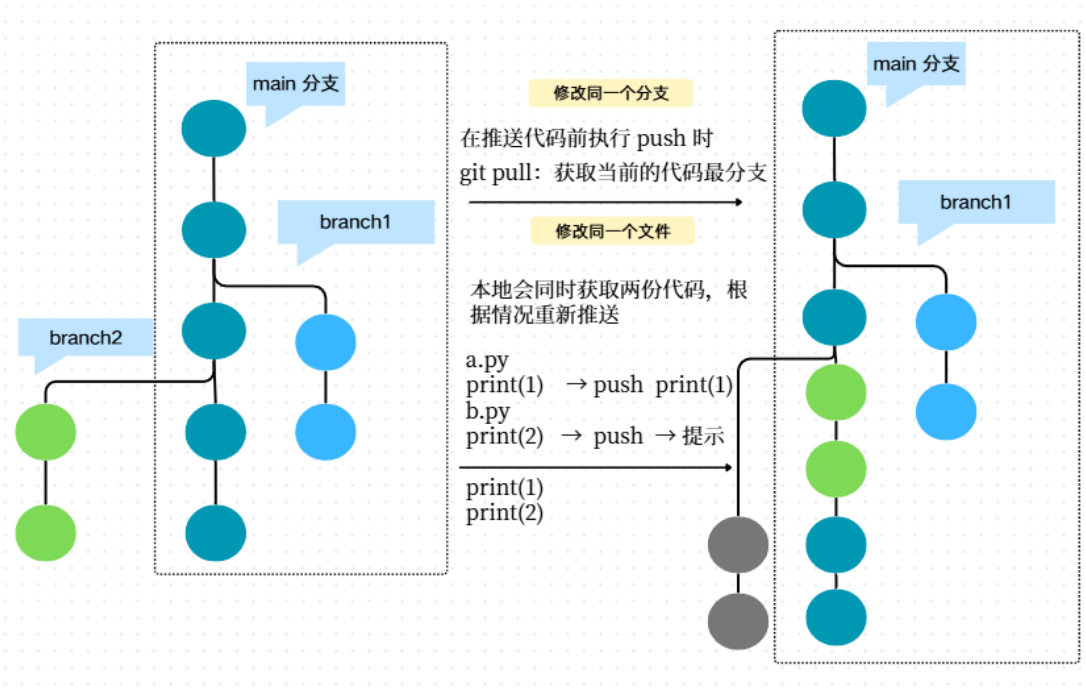
## Git 代码操作流程图



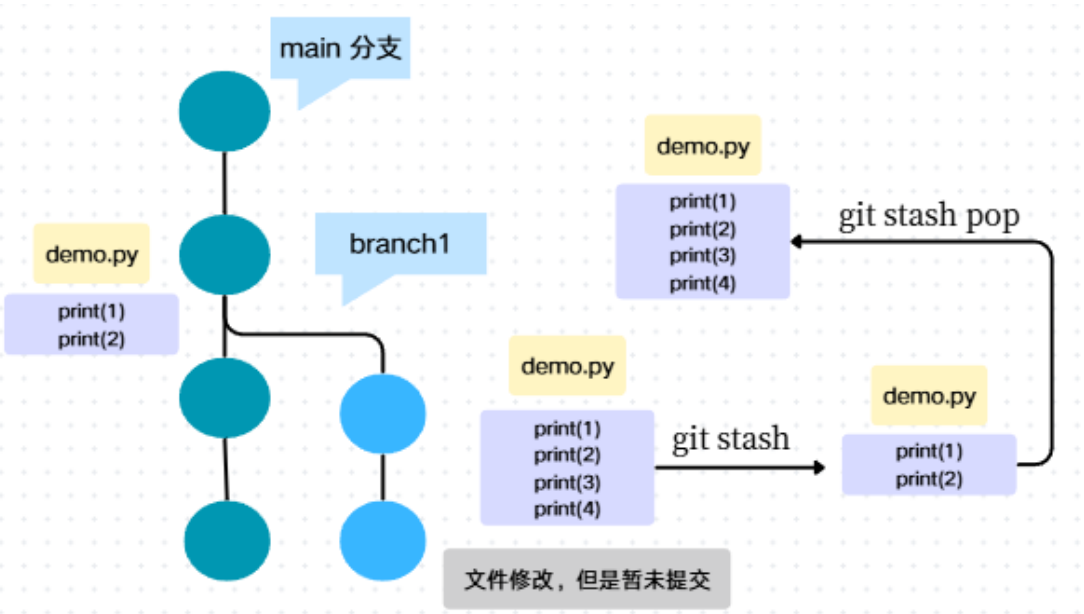

| | |
|---|---|
| workspace | 工作区 |
| staging area | 暂存区/缓存区 |
| local repository | 版本库或本地仓库 |
| remote repository | 远程仓库 |

## 4.Git 分支管理

<table>
<tr><td rowspan="8">  </td><td>创建并切换分支</td><td>git checkout -b branch_name<br>git switch -b branch_name</td></tr>
<tr><td>切换分支</td><td>git checkout branch_name<br>git switch branch_name</td></tr>
<tr><td>查看分支</td><td>git branch<br>git branch - r 远程分支<br>git branch -a 本地和远程</td></tr>
<tr><td>推送分支至远程</td><td>git push origin <branchname></td></tr>
<tr><td>将其他分支合并到当前分支</td><td>git merge <branchname></td></tr>
<tr><td colspan="2"> 
</td></tr>
<tr><td>首次提交本地新建分支</td><td>git remote add [shortname] [url]</td></tr>
<tr><td>合并分支时当前分支落后</td><td>git pull(在同一个分支下修改)</td></tr>
</table>

| | |
|---|---|
| |  |
| 删除本地分支： | git branch -d <branchname> |
| 删除远程分支： | git push origin --delete <branchname> |

| git stash(分支状态储存) | |
|---|---|
| 保存当前工作目录中的未提交更改，并将其恢复到干净的工作区。 | |
| 保存当前更改 | git stash<br>git stash push |
| 应用最近保存的更改，并删除 stash | git stash pop<br>存在多个保存时：git stash pop stash@{1} |

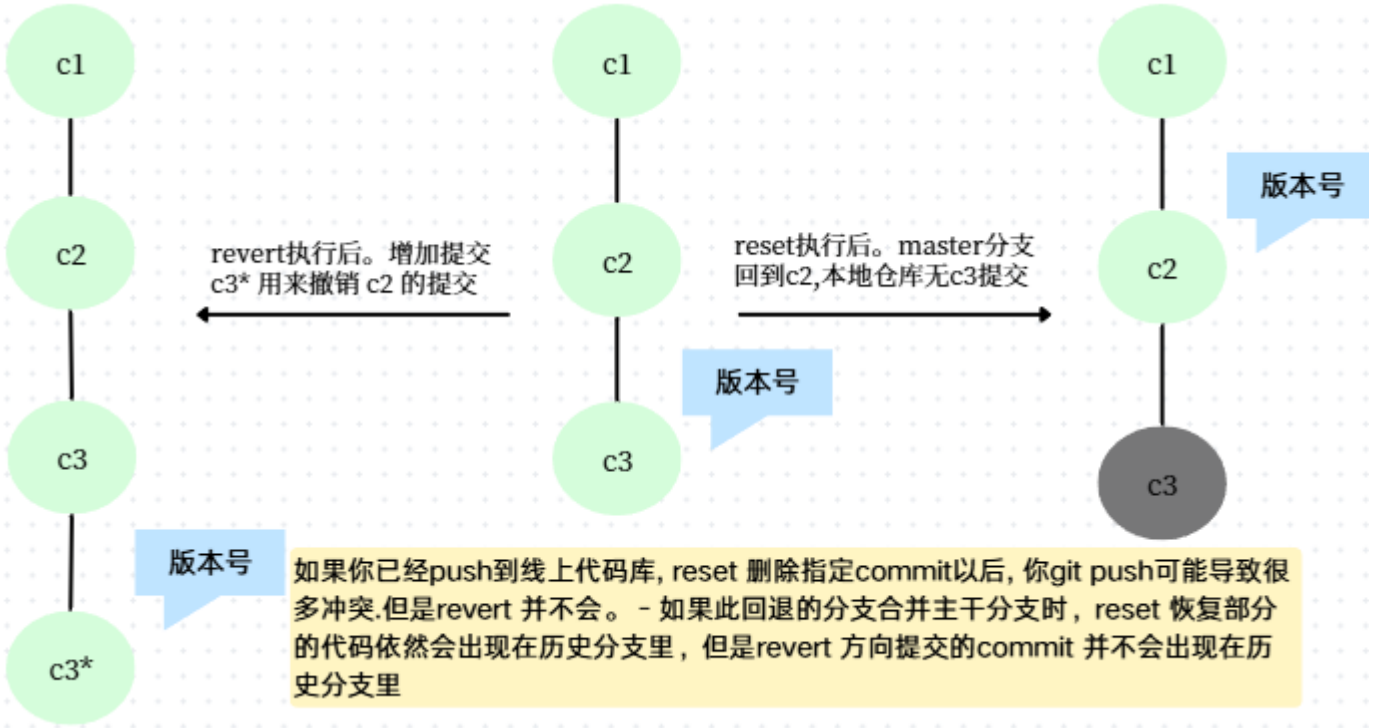



| | |
|---|---|
| 列出所有保存的更改 | git stash list |
| 将当前工作空间的状态保存并指定一个日志信息 | git stash save “信息” |
| 创建并切换到一个新分支来读取指定的存储<br>stashName:<br>存储的名称，默认情况下读取 stash 堆栈中栈顶的存储 | git stash branch {branchName}<br>[stashName] |
| 应用最近一次的 stash 更改 | git stash apply |
| 应用 stash 指定条目的更改 | git stash apply stash@{2} |
| 删除指定的 stash 条目 | git stash drop stash@{1} |
| 清除所有 stash 条目 | git stash clear |
| 交互式选择要暂存的工作区修改块 | git stash push -p<br>git stash -p |

| git stash | git commit |
|---|---|
| 保存当前工作目录中的未提交更改，并将其恢复到干净的工作区。 | 创建一个新的提交对象，永久保存更改。切换分支时，如果更改已提交，它们会存在于该分支的历史中 |
| 切换分支时 | |
| 使用 git stash，更改被隐藏，分支切换干净，之后可以恢复更改 | 使用 git commit，更改成为分支的一部分，切换分支不会影响这些更改 |

## 4.Git 提交与记录

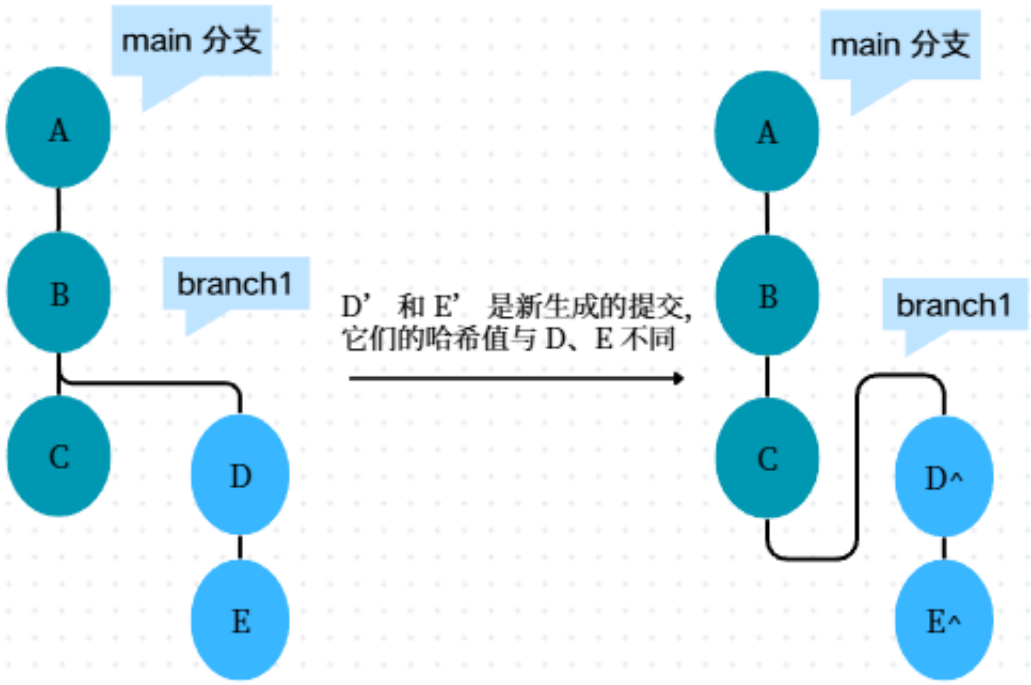
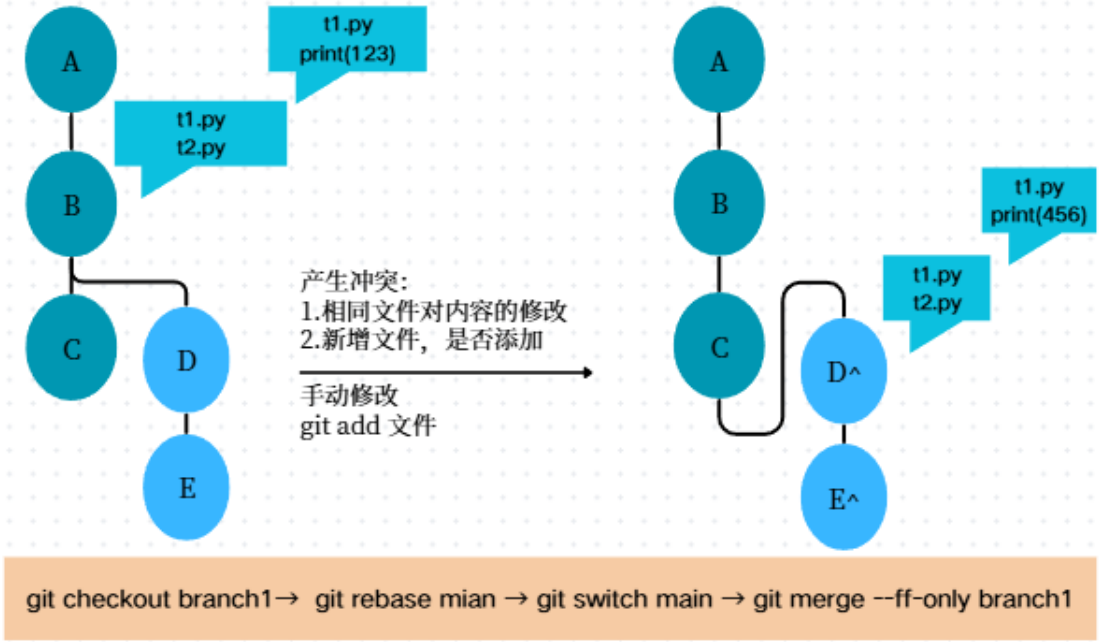
| | |
|---|---|
| 查看提交记录 | git log |
| 简洁版本 | git log -oneline |
| 查看历史中什么时候出现了分支、合并 | git log -graph |
| 逆向显示所有日志 | git log --reverse --oneline |
| 查找指定用户提交的日志 | git log --author=Linus |
| reset（重置当前分支到特定提交（对本地操作）） | |
| 用于本地撤销提交，快速丢弃错误修改,快速回退到某个指定的版本 | |
| 所有修改（包括已暂存和未暂存的）都保留在工作区，且**自动进入暂存区**（相当于执行了 git add）<br>撤销上次提交，但想保留修改，重新组织提交内容 | git reset --soft <commit> |
| 所有修改都保留在工作区，但**不暂存**（相当于没执行 git add）<br>回退**提交历史**和**暂存区**，但工作区不变 | git reset --mixed <commit> |
| 所有未提交的修改（包括已暂存和未暂存的）都会被**彻底删除**<br>回退**提交历史**、**暂存区**和**工作区**，三者完全同步到指定提交 | git reset --hard <commit> |
| 回退所有内容到上一个版本 | git reset HEAD^ |
| 回退 test.txt 这个文件的版本到上一个版本 | git reset HEAD^ test.txt |
| 向前回退到第 3 个版本 | git reset HEAD~3 |
| revert | |
| 用于公共分支撤销已推送的提交，安全且不破坏历史 | |
| 撤销某次提交 | git revert <commit> |
| Reset 和 revert 区别 | |




| | |
|---|---|
| **查看历史操作记录** | git reflog |

| git tag 标签（本质是某一个版本提交的别名，便于理解、软件发布） | |
|---|---|
| 创建轻量标签 | git tag v1.0 |
| 创建附注标签 | git tag -a v1.1 -m '附注标签' |
| 指定版本创建标签 | git tag -a v1.0 版本号 -m '第一个版本' |
| 删除标签 | git tag -d v0.1 |
| 共享标签 | git push origin v1.5 |

| git add -p | |
|---|---|
| 交互式暂存会逐步提示某个文件是否需要暂存 | y：暂存当前块<br>n：跳过当前块<br>s：拆分当前块<br>e：手动编辑当前块<br>q：退出暂存<br>a:当前以及之后所有修改暂存<br>d:忽略之后的所有修改 |
| Git rebase 变基 | |

| |
|---|
| 将一个分支上的更改移到另一个分支之上。它可以帮助保持提交历史的线性，减少合并时的冲突 |
|  |
| 文件存在冲突 |
|  |
| 变基过程中：fatal: It seems that there is already a rebase-merge directory, and I wonder if you are in the middle of another rebase...(编辑过程中执行其他操作，add，commit 等均触发 ) |

| | |
|---|---|
| 继续根据原来的记录变基 | git rebase –continue |
| 跳过当前的提交 | git rebase --skip |
| 放弃磁场变基操作 | git rebase --abort |
| 强制删除变基状态 | rm -fr .git/rebase-merge |
| | |
| 变基当前分支到指定分支： | git rebase <branchname> |

交互式变基：允许你在变基过程中编辑、删除或合并提交

执行 git rebase -i HEAD~4 ：获得前 4 条记录

```
pick ecd1f47 交互变基测试
pick 8811ef1 交互变基测试2
pick 31142f3 交互变基测试3
pick aea4696 交互变基测试4
```

| | |
|---|---|
| 保留提交 | pick |
| 修改提交信息 | reword |
| 编辑提交 | edit |
| 将当前提交与前一个提交合并 | squash |
| 将当前提交与前一个提交合并，不保留提交信息 | fixup |
| 删除提交 | drop |

示例：1、合并第二个提交至第一个 2、重写第三个提交信息 3、删除第四个提交

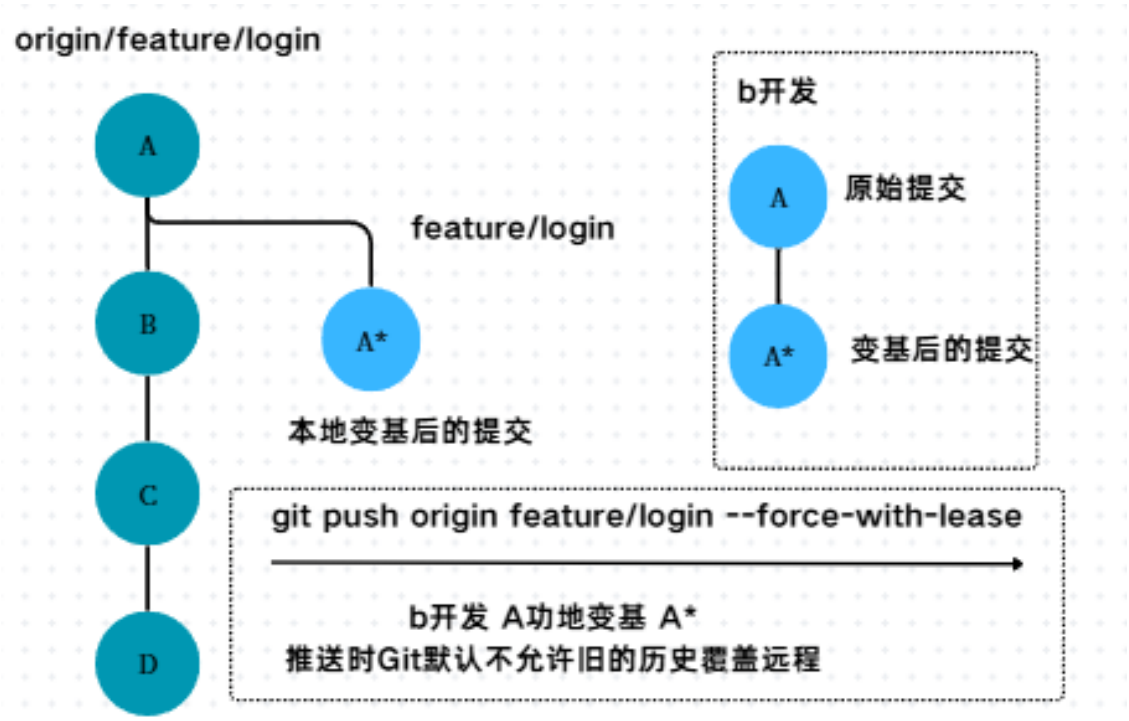
pick ecd1f47 交互变基测试

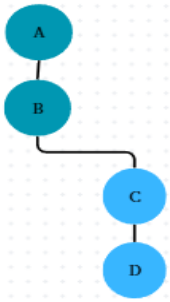
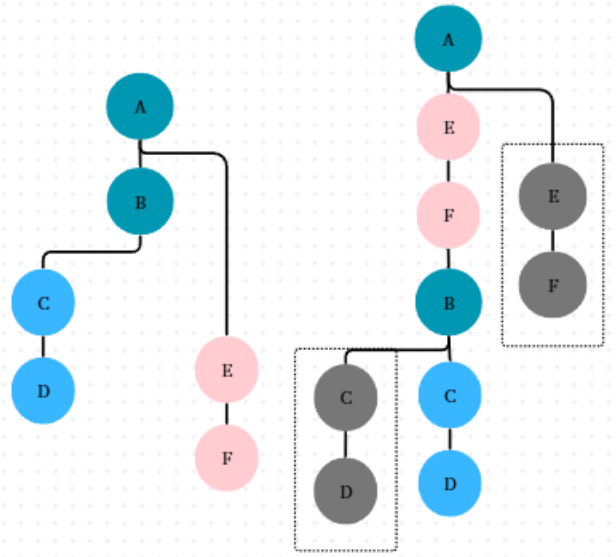
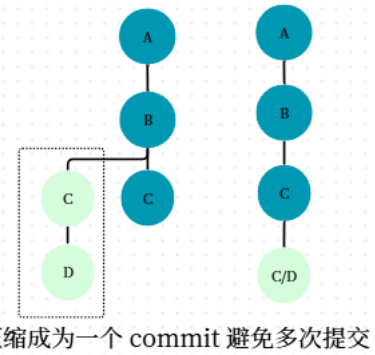
squash 8811ef1 交互变基测试 2

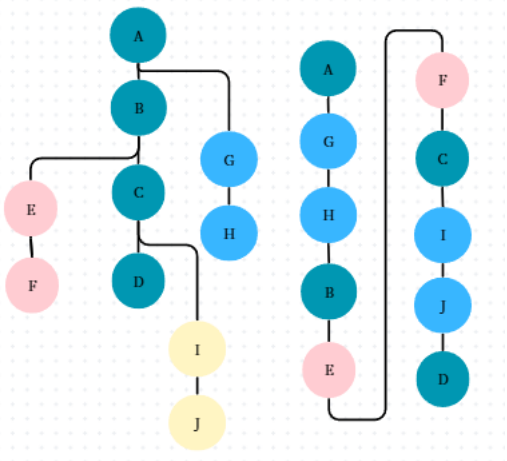
reword 31142f3 交互变基测试 3

drop aea4696 交互变基测试 4

| | |
|---|---|
| 远程分支变基 | Git pull origin <branchname>--force-with-lease |



<table>
<tr><td colspan="3">Git merge:<br>把这两个分支父节点本身及它们所有的祖先都包含进来。</td></tr>
<tr><td>fast-forward merge<br>快进合并（合并策略）</td><td>目标分支在当前分支创建后没有新提交，两个分支的提交历史是线性的，快进到当前提交，不会创建合并提交</td><td> 
</td></tr>
<tr><td>Three-way merge<br>（三方合并）</td><td>当目标分支和当前分支都有新提交，且无法进行快进合并时（分支历史出现分叉），寻找共同祖先提交，并且创建新的合并提交，记录合并操作</td><td> 
</td></tr>
<tr><td>Squash merge（压缩合并）</td><td>希望将某分支的多个提交压缩为一个提交后再合并到目标分支，保持主分支历史简洁（不保留原分支的提交历史）</td><td> 

压缩成为一个 commit 避免多次提交</td></tr>
<tr><td>Octopus merge<br>(章鱼合并)</td><td>一次性合并多个分支到目标分支，并后会创建一个有多个父提交的合并节点</td><td> 
</td></tr>
</table>

| | |
|---|---|
| | |
| **Git Cherry-Pick**<br>拣选提交：允许你选择特定的提交并将其应用到当前分支 | |
| 将单个提交应用至当前分支 | git cherry-pick <commit> |
| 复制指定分支最新提交 | git cherry-pick <branchName> |
| 一次性复制多个提交 | git cherry-pick <HashA> <HashB> |
| 复制一段连续提交 | git cherry-pick A..B |
| 跨仓库复制提交 | 1. git remote add target <gitUrl> ：添加仓库<br>2. git fetch target ：拿到远程分支的提交<br>3. git cherry-pick <commitHash>：通过 cherry-pick 复制到本地当前分支 |
| 先解决冲突并将文件 git add 至暂存区后，再执行本命令继续 Cherry-pick 剩余操作 | git cherry-pick --continue |
| 放弃这次操作，工作区回到 cherry-pick 前的状态，仿佛未曾进行过 cherry-pick | git cherry-pick --abort |
| 退出 cherry-pick，不恢复到操作前的状态，已合并到工作区或暂存区的修改会保留 | git cherry-pick --quit |

| Submodule 子模块 | |
|---|---|
| Git Submodule 是一种让 Git 仓库作为另一个 Git 仓库的子目录的方法。这在管理大型项目时非常有用，特别是当项目依赖于其他项目或库时 | |
| 创建子模块，添加关联 | git submodule add <ssh_url> <存放路径><br>存放路径：必须为空、或不存在、或存在 commit 的 Git 仓库 |
| 查看子模块 | git submodule<br>git submodule status 详细输出 |
| 更新子模块 | git submodule update –remote<br>将子模块更新到远程仓库的最新提交 |
| 克隆包含子模块的项目 | git clone https://github.com/demo.git assets --recursive<br>递归克隆子模块所有内容<br>若无--recursive 子目录为空 |
| 递归更新整个项目的子模块 | git submodule foreach git pull<br>遍历所有的子模块，并执行 git pull 命令 |
| 删除子模块 | git rm --cached subModulesA<br>从 git 中移除子模块的引用和配置 |
| | rm -rf subModulesA<br>删除子模块文件夹本身 |

| Other | |
|---|---|
| git remote prune | 从远程仓库中删除已经不存在的分支对应的本地 remote-tracking 分支 |
| git archive<br>将指定分支或者提交导出为一个压缩文件（如 zip） | |
| 创建一个 tar 归档文件 | git archive --format=tar --output=project.tar master |
| 创建一个 zip 归档 | git archive --format=zip --output=project.zip master |
| git bundle<br>将指定的分支或者提交打包为一个二进制文件 | |
| 创建一个 Bundle：假设你想把 master 分支打包成一个 bundle 文件 | git bundle create repo.bundle master |
| 在另一台机器上你可以将其视为一个远程仓库 | git bundle verify repo.bundle 检查 bundle 文件是否包含引用<br>git clone repo.bundle -b master /path/to/new/repository 可以实现远程克隆拉取<br>git pull /path/to/repo.bundle master 在已有仓库拉取更新 |
| | |
| Git reflog | 查看提交的日志（恢复本地的错误操作） |
| git commit --fixup | 将指定的提交标记为修复提交，在后续的 rebase 中可以自动合并 |
| git show-branch | 显示多个分支的合并历史 |
| git<br>diff --name-only --diff-filter=U | 显示合并时发生冲突的文件 |
| git diff | 打开冲突文件 |
| git clean -xdf | 清理工作区中未被版本控制的所有文件和文件夹包括忽略文件 |
| git commit --amend | 修改最后一次提交 |
| git diff --word-diff | 显示文件更改的详细信息，以单词为单位进行比较 |